

Spectrum Sensing System

February 7, 2026

Cognitive Radio: Cognitive radio (CR) allows devices to intelligently detect available channels in a wireless spectrum and change transmission parameters accordingly. The main goals of CR are to improve spectrum utilization and to enable dynamic spectrum access. CR can adapt their transmission parameters based on the surrounding environment, including frequency, power, and modulation schemes. CR includes Spectrum-sensing technology that enhances spectrum use efficiency by continuously monitoring and analyzing the radio spectrum

Cognitive Radio Spectrum Sensing: Spectrum sensing is the process of detecting the presence or absence of signals in a given frequency spectrum, enabling the identification of unused frequency bands (spectrum holes) that can be utilized by secondary users without interfering with primary users. The high-level tasks of cognitive radio spectrum sensing include:

- *Detection:* Accurately identifying whether a specific frequency band is occupied or vacant.
- *Classification:* Classifying the type of signals present in the spectrum to determine the nature of the primary users.
- *Localization:* Identifying the location of primary users to ensure that cognitive radios do not interfere with them.
- *Decision Making:* Making real-time decisions about when and how to access the spectrum based on the sensing results.

Tasks of Spectrum Sensing Detection

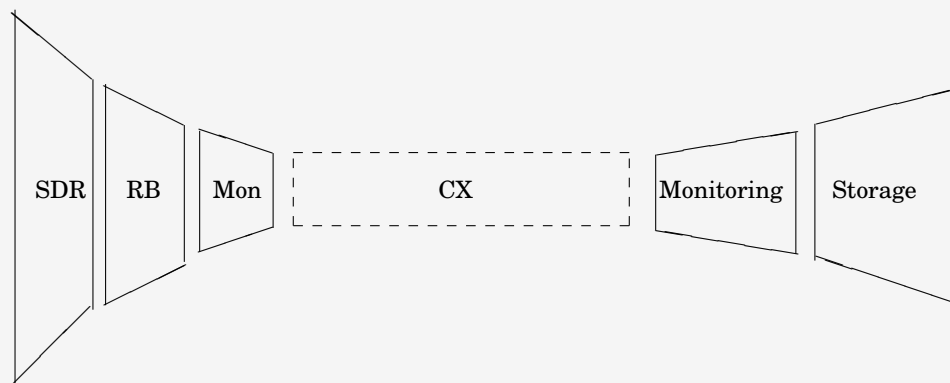
- *Identification of Available Spectrum:* The primary objective is to ascertain the presence of vacant frequency bands, thereby enabling secondary users to utilize these resources without causing interference to the licensed primary users.
- *Interference Mitigation:* Spectrum sensing must accurately detect the presence of primary users to prevent any disruption of their communication, thus ensuring the protection of their established services.
- *Spectrum Efficiency Enhancement:* Spectrum sensing plays a crucial role in enabling dynamic spectrum access, which in turn optimizes spectrum utilization and improves resource allocation efficiency.

Algorithm Selection and Implementation Framework

This system addresses three fundamental challenges in CR applications:

- Detection and identification of available spectrum in low SNR conditions
- Robust operation under noise uncertainty
- Accurate modulation recognition for enhanced spectrum utilization

The HackRF One SDR provides a cost-effective, wideband hardware platform (~ 1 MHz-6 GHz, up to 20 MS/s, half-duplex) that is well suited for implementing and testing spectrum monitoring and spectrum sensing algorithms in realistic RF environments.



output SDR data $\mathcal{X}$, Dimension $D \mapsto$ RB data $\mathcal{X}'$, $D' \mapsto$ Mondata $\mathcal{X}''$, $D'' \mapsto$ CX

PSD Estimation for Spectrum Sensing

This task involves estimating the distribution of power across different frequency components of a signal.

1. **Spectral Parameter Estimation.** Key features derived from the PSD include:

- Power levels at specific frequencies, `01ePSDHistStat` •
- Peak frequencies (locations of maximum power),
- Bandwidth (range of occupied frequencies),
- Center frequency (mean spectral centroid).
- Signal-to-Noise Ratio

2. **Quantitative Reconstruction of PSD.** This involves reconstructing the original PSD from noisy or undersampled data using techniques such as interpolation, smoothing, or model-based estimation.

3. **Demodulation Support.** PSD estimation assists in identifying carrier frequencies and modulation schemes, facilitating demodulation of the received signals.

Signal Detection

This task focuses on determining the presence of informative content within the received signal.

1. **Detection of Signal Presence (Thresholding).** A threshold is applied to a test statistic (e.g., energy) to decide whether a signal is present.
2. **Symbol Detection.** Once signal presence is confirmed, the next step is identifying the transmitted symbols, such as bits in digital communication systems.

Identification and Classification of Signals

This task involves characterizing signals based on observed features.

1. **Modulation Classification.** Determining the modulation format (e.g., AM, FM, QAM, PSK) is critical for enabling correct demodulation and decoding.
2. **Spatio-Temporal Identification of Spurious Signals.** This involves localizing and characterizing unwanted signals across both spatial and temporal domains.
3. **Spatio-Temporal Identification of Interference.** Identifying and profiling interference sources that affect the desired signal, considering their spatio-temporal evolution.

Channel State Estimation

Characterizing the wireless channel is essential for compensating for impairments such as fading or noise.

1. **Channel Impulse Response (CIR).** Estimation of how the channel distorts the temporal profile of the signal.
2. **Channel Frequency Response (CFR).** Estimation of the channel's effect on signal amplitude and phase across frequency.

Decision-Making

Based on the outcomes of the previous steps, appropriate decisions are made to complete the communication or control loop.

1. Decoding of the received data.
2. Choosing an optimal action in adaptive or control-oriented systems.
3. Interference identification and mitigation strategies.

System Architecture

Processing Pipeline

acquisition → PSD → energy detection → advanced detection → modulation recognition → integration & optimization

The spectrum sensing system follows a multi-stage processing pipeline:

1. *Signal Acquisition*: HackRF One captures IQ samples
2. *Preprocessing*: Filtering, decimation, AGC
3. *PSD Estimation*: Welch/Multitaper methods
4. *Noise Floor Estimation*: Dynamic noise power calculation
5. *Feature Extraction*: Statistical and spectral features
6. *Spectrum Sensing Decision*: Adaptive thresholding
7. *Modulation Recognition*: ML-based classification (if signal present)

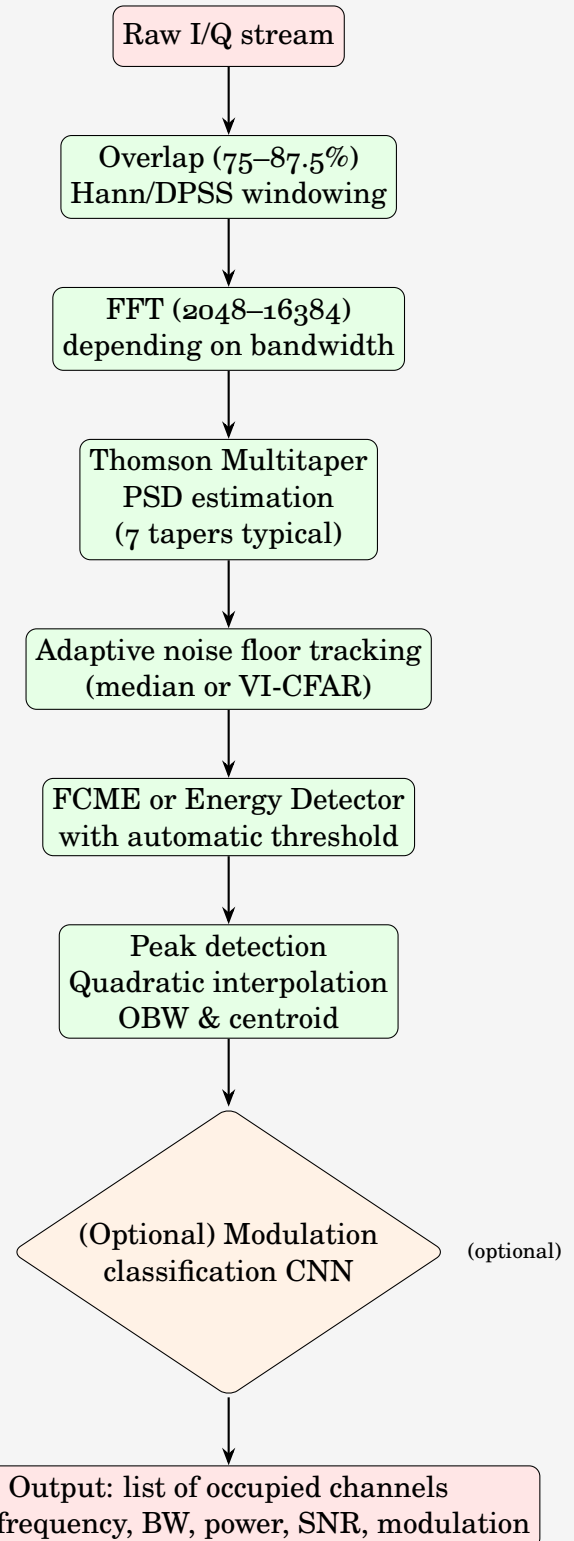
Development Phases

Phase 0: Signal Acquisition & Preprocessing
Objectives:

- ✗ Standardized Data Acquisition,
- ✗ Filtration, decimation and sub-banding
- ✗ streaming buffers, threading model multi-threading, GPU acceleration

Deliverables:

- Modular and reproducible code scripts for preprocessing, and filtering
- HackRF configuration profiles and data format/metadata specification
- Streaming buffer + threading implementation with basic performance and calibration report



*Phase 1: PSD Estimation & Visualization***Objectives:**

- ✓ Implementation of the methods for PSD estimation
- ✗ Real-time spectrum waterfall display
- ✗ Noise floor estimation and tracking

Deliverables:

- Functional PSD estimation module
- Visualization interface
- Initial performance benchmarks

*Phase 2: Energy Detection Implementation***Objectives:**

- Adaptive thresholding (CFAR algorithm)
- Signal presence detection with configurable P_{fa}
- Performance evaluation across frequency range

Deliverables:

- Energy detection module
- Threshold adaptation algorithm
- ROC curves and performance metrics

*Phase 3: Advanced Detection***Objectives:**

- Cyclostationary feature detection implementation
- Comparative performance analysis
- Hybrid detection strategy development

Deliverables:

- Cyclostationary detector
- Performance comparison report
- Optimized hybrid approach

*Phase 4: Modulation Recognition***Objectives:**

- Feature extraction pipeline development
- Train CNN on synthetic/captured dataset
- Real-time classification integration

Deliverables:

- Feature extraction module
- Trained classification model
- Real-time recognition system

Phase 5: System Integration & Optimization

Objectives:

- End-to-end testing and validation
- Performance optimization
- User interface development

Deliverables:

- Fully integrated system
- Comprehensive documentation
- User manual and tutorials

Implementation Issues

PHASE 0: OPTIMIZATION FOR LOW SNR

Phase 0 establishes the acquisition and preprocessing backbone required to make the HackRF-based spectrum monitor robust in low-SNR regimes. The goal is to ensure that all subsequent PSD estimation and detection stages operate on well-calibrated, statistically stable I/Q data, while respecting real-time constraints at up to 10 MS/s.

Implementation focus. The following components are implemented in Phase 0 as the foundation for low-SNR optimization:

- ✓ *Modular acquisition and preprocessing scripts* Reusable Python and/or GNU Radio blocks for HackRF I/Q capture, decimation, and basic bandpass filtering, providing a clean and well-documented interface to the PSD and detection modules.

(A) HackRF baseband filter bandwidth (practical selection rule). HackRF One does *not* implement a true RF band-pass filter; instead it exposes a selectable *baseband/channel* filter (MAX2837) around the tuned center frequency. Let B_{ROI} denote the target ROI (occupied) bandwidth. Choose the baseband filter bandwidth

$$B_{\text{BBF}} \in \{1.75, 2.5, 3.5, 5, 5.5, 6, 7, 8, 9, 10, 12, 14, 15, 20, 24, 28\} \text{ MHz}, \quad (1)$$

as the *smallest available* value satisfying

$$B_{\text{BBF}} \geq (1 + g) B_{\text{ROI}}, \quad (2)$$

where the guard factor g depends on the task:

- *Energy/occupancy screening*: $g \approx 0.10\text{--}0.30$ (more tolerant to leakage, prioritize robustness).
- *Demodulation / feature extraction*: $g \approx 0.05\text{--}0.15$ (tighter passband, less adjacent energy).

(B) Digital band-pass filtering for HackRF (DSP-side). Since HackRF provides a tunable complex baseband stream, the most efficient way to realize a *digital band-pass* around an RF region-of-interest (ROI) is:

frequency shift (mix) $\rightarrow$ complex low-pass $\rightarrow$ decimate.

This is equivalent to band-pass filtering in the original RF domain, but cheaper and more stable for real-time pipelines.

Shift-LPF-Decimate model. Let $x[n]$ be the complex baseband at sample rate F_s (Hz). Let f_0 be the ROI center offset (Hz) relative to the tuned center. First, shift the ROI to DC:

$$x_0[n] = x[n] e^{-j2\pi \frac{f_0}{F_s} n}. \quad (3)$$

Then apply a complex low-pass FIR $h[n]$ and decimate by an integer factor D :

$$y[n] = \sum_k h[k] x_0[nD - k]. \quad (4)$$

After decimation, the effective sample rate is $F'_s = F_s/D$.

Passband, guard, and transition width. Let B denote the desired occupied bandwidth (Hz). Choose the low-pass *passband edge*

$$f_p = \frac{B}{2} + g, \quad (5)$$

where the guard g (Hz) depends on the task:

- *Energy/occupancy screening*: $g \approx 0.10B$ to $0.30B$.
- *Demodulation/feature extraction*: $g \approx 0.05B$ to $0.15B$.

Choose a transition width Δf (Hz):

- *Real-time / lighter FIR*: $\Delta f \approx 0.10B$ to $0.25B$.
- *Strong adjacent blockers (near-far)*: $\Delta f \approx 0.02B$ to $0.10B$ (more taps).

The stopband edge is $f_s = f_p + \Delta f$.

Default magnitude specs (robust on SDR captures). A practical baseline is:

$$R_p \in [0.1, 0.5] \text{ dB} \quad (\text{use } 0.2 \text{ dB as default}), \quad (6)$$

$$A_s \geq 60 \text{ dB} \quad (\text{use } 70\text{--}80 \text{ dB for near-far}). \quad (7)$$

FIR length sizing (Kaiser rule of thumb). For a Kaiser-window FIR, an approximate length is

$$N \approx \frac{A_s - 8}{2.285 \Delta\omega}, \quad \Delta\omega = 2\pi \frac{\Delta f}{F_s} \Rightarrow N \approx \frac{(A_s - 8) F_s}{2.285 \cdot 2\pi \Delta f}. \quad (8)$$

If N becomes too large at high F_s , use multistage decimation:

- Decimate by 2 repeatedly with halfband filters (cheap),
- Then apply one final sharp low-pass at the reduced rate for the desired B .

Two practical presets (drop-in).

- *Preset 1 (occupancy / PSD features)*: $R_p = 0.2 \text{ dB}$, $A_s = 60 \text{ dB}$, $\Delta f = 0.15B$, $g = 0.20B$.
- *Preset 2 (near-far / strong adjacent interferers)*: $R_p = 0.1 \text{ dB}$, $A_s = 80 \text{ dB}$, $\Delta f = 0.05B$, $g = 0.10B$, with multistage decimation recommended when starting from $F_s \geq 10 \text{ MS/s}$.

Wideband scanning default (e.g., $F_s = 10 \text{ MHz}$). A stable, common choice is $B_{\text{BBF}} \approx 7\text{--}8 \text{ MHz}$, because the HackRF tools default the baseband filter to a value *no larger than* $\approx 0.75 F_s$ after setting the sampling rate; for $F_s = 10 \text{ MHz}$, $0.75 F_s = 7.5 \text{ MHz}$, hence selecting the nearest available step (7 or 8 MHz) is a sensible default.¹

Important low-sample-rate constraint. Using $F_s < 8 \text{ MHz}$ is *not recommended* because (i) the MAX5864 ADC/DAC is not specified below 8 MHz, and (ii) the MAX2837 baseband filter has a minimum bandwidth of 1.75 MHz, which is too wide to sufficiently suppress adjacent-spectrum energy before aliasing at very low F_s (e.g., 2 MHz). A standard

¹<https://github.com/mossmann/hackrf/issues/369>

workaround is to sample at 8 MHz (minimum recommended), enable the minimum 1.75 MHz baseband filter, and then decimate digitally (e.g., 4:1) with a sharp complex low-pass to reach an effective lower rate.²

- ✓ **HackRF configuration profiles** Standardized configuration files (e.g., YAML/JSON) that specify center frequency, sample rate, gain settings, and DC-offset mode for each monitored band, ensuring reproducible experiments under varying SNR conditions.

(C)Per-band HackRF configuration (sample rate, gains, DC handling). HackRF One is a *single-channel* SDR; thus, *per monitored band* means maintaining a configuration profile that is applied each time the receiver retunes. For each band k , define the profile

$$\mathcal{P}_k = (f_{c,k}, F_{s,k}, B_{\text{BBF},k}, \text{AMP}_k, \text{LNA}_k, \text{VGA}_k, \pi_k^{\text{DC}}), \quad (9)$$

where $f_{c,k}$ is the tuned center frequency, $F_{s,k}$ the complex sample rate, $B_{\text{BBF},k}$ the HackRF baseband/channel filter bandwidth, $(\text{AMP}_k, \text{LNA}_k, \text{VGA}_k)$ the gain settings, and π_k^{DC} the DC-offset policy.

1) Sample rate selection $F_{s,k}$. Choose $F_{s,k}$ to cover the *instantaneous* bandwidth you want to capture for band k (scan chunk), while avoiding very low sample rates. HackRF documentation recommends *not* operating below 8 MHz because the MAX5864 ADC/DAC is not specified below 8 MHz and, additionally, the MAX2837 baseband filter has a minimum bandwidth of 1.75 MHz, which is too wide to reliably suppress adjacent energy before aliasing at very low F_s . A standard workaround for narrowband monitoring is to sample at 8 MHz and then decimate digitally to the desired effective rate.³

- **Wideband monitoring:** typical stable choices are $F_{s,k} = 10$ MHz (balanced) or 16–20 MHz if the host I/O and DSP budget allow.
- **Narrowband monitoring:** set $F_{s,k} = 8$ MHz and perform *digital* low-pass filtering and decimation (multistage if needed).

Command-line setting.

```
# Example: 10 MS/s
hackrf_transfer -f <fc_hz> -s 10000000 ...
```

```
# Example: 8 MS/s (then decimate in DSP)
hackrf_transfer -f <fc_hz> -s 8000000 ...
```

2) Gain settings per band $(\text{AMP}_k, \text{LNA}_k, \text{VGA}_k)$. HackRF RX gains consist of: (i) an optional RF amplifier (AMP, on/off, ≈ 11 dB), (ii) an IF gain (LNA, 0–40 dB in 8 dB steps), and (iii) a baseband gain (VGA, 0–62 dB in 2 dB steps).⁴ A practical per-band calibration procedure is:

²https://hackrf.readthedocs.io/en/latest/sampling_rate.html

³https://hackrf.readthedocs.io/en/latest/sampling_rate.html

⁴https://hackrf.readthedocs.io/en/latest/hackrf_tools.html

1. *Initialize* with a conservative default (commonly used starting point):

$$(\text{AMP}_k, \text{LNA}_k, \text{VGA}_k) = (0, 16, 16).$$

2. *Adjust* LNA and VGA *roughly together* to lift the ROI above the noise floor while avoiding distortion/spurious products that increase with gain.⁵
3. *Enable* AMP only for weak-signal bands and only if overload in crowded conditions is not observed.⁶

Command-line setting.

```
# -a: AMP (0/1), -l: LNA (0..40, step 8), -g: VGA (0..62, step 2)
hackrf_transfer -f <fc_hz> -s <Fs> -a 0 -l 16 -g 16 -r band_k.iq
```

3) DC-offset handling policy π_k^{DC} . A DC spike near 0 Hz is a common quadrature-sampling artifact and may appear prominently in FFT/PSD views. HackRF documentation recommends *offset tuning* as the most robust approach: tune slightly away from the exact ROI center so that the ROI does not lie at DC, then digitally frequency-shift back to baseband in software.⁷

Define the per-band DC policy as one of:

$$\pi_k^{\text{DC}} \in \{\text{offset_tune}, \text{dc_block}, \text{ignore}\}. \quad (10)$$

- **offset_tune (recommended default):** tune to $f_{c,k} = f_{c,k}^* + f_{\text{off},k}$ so the ROI is displaced from 0 Hz, then apply a digital mixer $e^{-j2\pi f_{\text{off},k}n/F_{s,k}}$ to re-center the ROI in DSP. Typical offsets are $f_{\text{off},k} \approx 200\text{--}500$ kHz for narrowband ROIs, or $f_{\text{off},k} \approx F_{s,k}/8$ for wideband scans.⁸
- **dc_block (software DC removal):** subtract the complex mean or apply a DC-blocking high-pass; this can degrade legitimate components near DC, so avoid when the ROI overlaps 0 Hz.⁹
- **ignore:** acceptable when the ROI is guaranteed to be far from DC (e.g., ROI selection excludes center bins).¹⁰

Recommended per-band profile template (drop-in). For each monitored band k , store:

$$\mathcal{P}_k = \left(f_{c,k}^*, f_{\text{off},k}, F_{s,k}, B_{\text{BBF},k}, \text{AMP}_k, \text{LNA}_k, \text{VGA}_k, \pi_k^{\text{DC}} \right), \quad (11)$$

where $f_{c,k}^*$ is the desired ROI center and $f_{\text{off},k}$ implements *offset_tune*. This makes the acquisition stage reproducible and allows band-specific tuning while preserving a uniform downstream DSP interface.

⁵https://hackrf.readthedocs.io/en/latest/hackrf_tools.html

⁶https://hackrf.readthedocs.io/en/latest/hackrf_tools.html

⁷<https://hackrf.readthedocs.io/en/latest/faq.html#why-is-there-a-spike-in-the-center-of-my-fft-spectrum>

⁸<https://hackrf.readthedocs.io/en/latest/faq.html#why-is-there-a-spike-in-the-center-of-my-fft-spectrum>

⁹<https://hackrf.readthedocs.io/en/latest/faq.html#why-is-there-a-spike-in-the-center-of-my-fft-spectrum>

¹⁰<https://hackrf.readthedocs.io/en/latest/faq.html#why-is-there-a-spike-in-the-center-of-my-fft-spectrum>

- ✓ *Streaming buffer and threading prototype* Design and implementation of a circular buffer and threading model capable of sustaining continuous streaming at up to 10 MS/s without buffer overflows, including basic monitoring and logging of dropped samples.

(D)Synchronizing concurrent DSP consumers to avoid HackRF overruns. When running two concurrent processing tasks (e.g., wideband PSD estimation and narrow-band audio reconstruction) from a single HackRF RX stream, the primary design goal is:

Never allow heavy DSP to block the USB/RX ingestion path.

HackRF streams continuously; if the host does not drain samples fast enough, RX overruns occur (typically observed as 0 overrun reports in streaming tools).¹¹

1) Producer–consumer architecture with a lock-free ring buffer. Implement a strict producer–consumer pipeline:

- *RX producer thread (highest priority)*: reads IQ blocks from HackRF and writes them into a preallocated ring buffer; it performs *no* expensive DSP.
- *Consumer A (PSD worker)*: consumes blocks and computes PSD/occupancy at a reduced cadence (loss-tolerant).
- *Consumer B (audio worker)*: consumes (near-)all blocks to preserve continuity (loss-intolerant).

If using libhackrf callbacks, keep the callback minimal and non-blocking, as blocking the callback risks overruns.¹²

Ring-buffer synchronization on sequence numbers. For each RX block, attach metadata:

$$\text{Block} = (\text{seq}, t_{\text{host}}, \mathbf{x} \in \mathbb{C}^{N_b}),$$

where seq increments monotonically per block. Let the ring have R slots and map

$$\text{slot} = \text{seq} \bmod R.$$

Each consumer maintains its own read pointer $\text{read_seq}^{(m)}$. A consumer is *behind* if

$$\text{write_seq} - \text{read_seq}^{(m)} \geq R,$$

which indicates lost data for that consumer (ring wrap).

2) Explicit drop and priority policy (loss-tolerant vs. loss-intolerant). Since HackRF cannot apply backpressure, overload must be handled by *selective dropping*:

- *Audio path*: treat as *continuity-critical* (avoid drops; reduce workload or F_s if needed).
- *PSD path*: treat as *loss-tolerant* (skip windows/frames under load).

¹¹https://hackrf.readthedocs.io/en/latest/hackrf_tools.html

¹²<https://github.com/greatscottgadgets/hackrf/blob/master/host/libhackrf/src/hackrf.c>

Implement high/low-watermarks on ring occupancy. For example, if ring fill exceeds $\eta_{hi} \approx 80\%$,

- reduce PSD update rate (compute fewer PSDs/sec),
- drop PSD blocks until occupancy falls below $\eta_{lo} \approx 50\%$,
- only then consider reducing F_s or simplifying the audio chain.

3) Share front-end DSP once, then branch. Avoid duplicating heavy processing on the full-rate stream. Use a shared lightweight front-end, then branch:

$$\text{RX} \rightarrow (\text{optional DC handling}) \rightarrow \text{channelize / DDC+decimate} \rightarrow \begin{cases} \text{PSD worker} \\ \text{audio worker} \end{cases}$$

For narrowband tasks, frequency shift $\rightarrow$ low-pass $\rightarrow$ decimate early so that subsequent demodulation runs at a low effective rate. For narrow effective rates, HackRF documentation recommends sampling at 8 MHz (minimum recommended region) and then digitally decimating (e.g., 4:1 to reach 2 MHz) instead of sampling below 8 MHz.¹³

4) Throughput-based buffer sizing. HackRF IQ samples are typically 8-bit I and 8-bit Q, i.e. ≈ 2 bytes per complex sample. The ingestion rate is therefore

$$\dot{B} \approx 2F_s \text{ bytes/s.}$$

To provide T seconds of cushion, size the ring buffer to

$$B_{\text{ring}} \approx 2F_s T \text{ bytes.}$$

Example: at $F_s = 10$ MHz, $\dot{B} \approx 20$ MB/s, so $T = 0.5$ s implies $B_{\text{ring}} \approx 10$ MB. This cushion absorbs transient scheduling jitter while the drop policy prevents sustained overload.¹⁴

5) Practical scheduling notes.

- Run the RX producer thread at the highest priority; pin it to a CPU core if possible.
- Preallocate buffers and avoid dynamic memory allocation in the hot path (RX callback/thread).
- Treat PSD computation as *adaptive*: decimate for PSD, reduce overlap, and decrease update rate under load.

Recommended synchronization contract (implementation summary).

- *RX producer*: writes `Block(seq, t_host, iq)` into ring; never blocks.
- *Audio consumer*: reads sequentially; if it falls behind, declare overload and reduce processing or F_s .
- *PSD consumer*: may read “latest available”; if behind, jump to newest seq and continue.

¹³https://hackrf.readthedocs.io/en/latest/sampling_rate.html

¹⁴https://hackrf.readthedocs.io/en/latest/hackrf_tools.html

Data format and metadata specification Formal specification of the on-disk I/Q format (e.g., complex64 or complex128) and associated metadata fields (timestamp, center frequency, bandwidth, gain configuration, calibration tag), so later phases can reliably consume Phase 0 recordings.

(E) Common on-disk I/Q (complex baseband) specifications. An on-disk I/Q recording typically stores a stream of complex baseband samples with no header. A reader must therefore know (or obtain from sidecar metadata) the sample layout, numeric datatype, byte order, scaling, and capture parameters (e.g., F_s and f_c).

Minimal contract (must be specified).

- *Layout*: interleaved complex samples $I[0], Q[0], I[1], Q[1], \dots$ (and channel interleaving if multichannel).
- *Datatype*: signed/unsigned vs. float, bit-width, and endianness (e.g., ci8, cu8, ci16_le, cf32_le).
- *Scaling*: mapping from integer codes to normalized amplitude (and DC offset convention for unsigned types).
- *Capture metadata*: at least sample rate F_s , center frequency f_c , gains, and start time / sample index.

Common raw encodings.

- *Signed 8-bit complex (ci8)*: two's-complement $I, Q \in [-128, 127]$, often normalized by $x/128$.
- *Unsigned 8-bit complex (cu8)*: $I, Q \in [0, 255]$, commonly mapped by $(x - 127.5)/127.5$.
- *Signed 16-bit complex (ci16_le)*: $I, Q \in [-32768, 32767]$, commonly normalized by $x/32768$.
- *Float32 complex (cf32_le)*: interleaved IEEE-754 float32 pairs (I, Q) .

Note on headers. Raw I/Q files are often headerless; if you need portability/reproducibility, store metadata in a sidecar JSON (e.g., SigMF-style) or another container format.

- ✓ *Calibration and normalization scripts* Standalone routines for IQ-imbalance correction, LO-leakage and DC-offset mitigation, and power normalization, complemented with before/after plots to document the effect of each correction and to verify stability over time.

(F) Standalone routines for IQ-imbalance correction, LO leakage and DC-offset mitigation, and power normalization

- IQ-imbalance correction (blind, widely-linear “circularity” compensation)
- LO leakage + DC-offset mitigation (mean/DC blocker + optional near-DC tone cancellation)
- Power normalization (RMS / dBFS target)

Assumptions: HackRF *hackrf_transfer* files are commonly raw, headerless, interleaved 8-bit signed IQ samples.

The “big spike in the center of the FFT” is typically DC offset / LO leakage.

Baseline calibrated dataset A small curated collection of HackRF recordings (noise-only, single-signal, and multi-signal scenes) acquired with the agreed profiles and calibration procedures, to be reused as a common benchmark for Phases 1–4.

Performance and resource usage report Quantitative assessment of CPU load, memory usage, and buffer-overflow rate for representative operating points (e.g., 10 MS/s, 1024–2048-point FFT, 50–75% overlap), including a brief discussion of real-time feasibility and headroom for more advanced detectors.

Quick-look validation notebook A Jupyter notebook that loads Phase 0 recordings, visualizes time-domain amplitude and spectrograms, and computes basic indicators (e.g., SNR estimates, approximate signal bandwidth) to validate acquisition quality before PSD and detection development.

Low-SNR enhancement strategies. Once acquisition and calibration are in place, the following strategies are used to improve detectability in low-SNR scenarios:

Hardware-Level Optimizations

Impedance Matching

- **Adaptive Impedance Matching**, Implement automatic impedance matching networks to minimize VSWR, Maximize power transfer between antenna and HackRF front-end
- **Manual Tuning Networks**, Add variable capacitors/inductors for bands of specific interest, Create custom matching networks for critical frequency ranges
- **Frequency-Adaptive Elements**, Implement electronically tunable elements (varactors, PIN diodes), Automatically adjust antenna electrical characteristics to match target frequencies, Create a software control loop that optimizes antenna tuning based on frequency of interest
- **Low-Noise Preamplifiers**, Add carefully selected LNAs close to antenna feed point Choose LNAs with noise figure lower than HackRF's native front-end, Consider bandpass filtering before amplification to prevent overload
- **Active Impedance Transformation**, Implement active circuits to transform impedance without losses associated with passive networks
- **Multi-antenna Setup**, Use 2-3 different antenna types simultaneously, Implement diversity combining in software, Switch between antennas based on frequency band and signal type

Control of front-end impairments and drift.

- IQ imbalance correction.
- LO-leakage removal.
- Thermal-drift monitoring and compensation (periodic re-calibration).
- Proper configuration and testing of HackRF DC-offset correction modes (`-dc-offset-mode`) for the bands and gains of interest.
- Enforcement of real-time architecture constraints (buffer sizes, thread priorities, I/O scheduling) to avoid overflows at 10 MS/s.

Pre-filtering and dynamic range control.

- Apply bandpass filtering to reject out-of-band interference and reduce the risk of ADC saturation from strong adjacent signals.
- Use automatic gain control (AGC) or carefully chosen static gains to normalize signal levels while avoiding clipping.
- Insert anti-aliasing filters (digital and, where necessary, analog) to prevent spectral folding into the band of interest.

Cooperative sensing (future enhancement). Although Phase 0 targets a single HackRF receiver, the framework anticipates future cooperative sensing with multiple HackRF units. In that case, local binary decisions D_i from each sensor can be fused using:

- OR rule: $D = \bigvee_{i=1}^N D_i$,
- AND rule: $D = \bigwedge_{i=1}^N D_i$,
- Weighted combination: $D = \sum_{i=1}^N w_i D_i \geq \lambda$,

where w_i are sensor weights and λ is a fusion threshold. This extension is deferred to later phases but is noted here to keep the low-SNR design compatible with multi-sensor deployments.

Enhanced Signal Processing Techniques

Frequency-domain processing and channelization.

- Cognitive Sampling. Dynamically adjust sampling parameters based on environmental conditions. Focus computational resources on frequency bands with higher probability of signal presence. Implement multi-rate sampling techniques to optimize for different signal types.
- Use polyphase filter banks to channelize the wideband input into narrower subbands.
- Process each subband independently to reduce computational load and improve frequency resolution where it is most needed.
- Align channel bandwidth with the target frequency resolution (e.g., 10–100 kHz) required by the sensing task.

Longer integration time.

- Increase the effective observation window to approximately 256–1024 ms (using sliding or overlapping windows).
- Trade latency for detection reliability, especially for weak or slowly varying signals.
- Improve effective SNR by a factor of $\sqrt{N}$ when averaging N statistically independent PSD estimates.

Temporal averaging and adaptive tracking. :

- Average PSD estimates across multiple epochs to reduce variance and stabilize the noise floor.
- Employ an exponential moving average for adaptive tracking:

$$\hat{P}(k) = \alpha P_{\text{current}}(k) + (1 - \alpha) \hat{P}(k - 1),$$

where $\alpha \in [0.1, 0.3]$ is a smoothing factor tuned to the dynamics of the environment.

Noise-uncertainty mitigation.

- Implement a two-stage detection strategy: (i) coarse energy detection for initial screening, and (ii) a refined detector (e.g., cyclostationary or eigenvalue-based) for ambiguous or low-SNR cases.

- Estimate noise power using the Forward Consecutive Mean Excision (FCME) algorithm to identify noise-only frequency bins. If N denotes the set of such bins and $M = |N|$, the estimated noise variance is

$$\hat{\sigma}_n^2 = \frac{1}{M} \sum_{k \in N} |X(k)|^2.$$

- Use the FCME-based noise estimate to adapt detection thresholds and maintain target false-alarm probabilities under noise uncertainty.
- Wavelet-Based Detection. Apply wavelet transforms for multi-resolution analysis of signals, Better time-frequency localization than FFT for transient or non-stationary signals, and can isolate weak signals in specific frequency bands where SNR might be marginally better
- Compressive Sensing. Reconstruct signals from fewer measurements than traditional Nyquist sampling requires. Particularly useful for detecting narrowband signals in wideband spectrum monitoring

Advanced ML/AI Approaches

- Deep Learning for Raw I/Q Processing. Train deep neural networks directly on I/Q samples for end-to-end detection. CNN or RNN architectures can learn to recognize patterns below traditional detection thresholds. Transfer learning from simulated or higher-SNR data to low-SNR scenarios
- Generative Adversarial Networks (GANs). Use GANs to synthesize clean signals from noisy observations. Train with paired noisy/clean signal examples. Particularly effective for known signal classes

Weaknesses and Gaps

- **Limited Real-World Testing Data:**

- While the report mentions a baseline calibrated dataset of HackRF recordings, there is no detailed discussion of real-world performance metrics or validation results under varying conditions (e.g., urban vs. rural environments).
- Lack of Empirical Validation> Lack of empirical data on detection accuracy, false alarm rates, or computational latency in low-SNR scenarios.

- **Hardware Limitations Not Fully Addressed:**

- The HackRF One's half-duplex nature and lack of true RF bandpass filtering are noted, but potential impacts on performance (e.g., aliasing at low sample rates) are not thoroughly analyzed with mitigation trade-offs.
- No discussion on hardware scalability or comparison with other SDR platforms (e.g., USRP) for higher performance needs.

- **Real-Time Feasibility Risks:**

- Real-time constraints at high sample rates (up to 10 MS/s) are acknowledged, but the report lacks a detailed risk assessment of potential bottlenecks (e.g., USB overruns, CPU/GPU limitations).
- The complexity of integrating machine learning (ML) models for modulation recognition in real-time is mentioned but not critically evaluated for feasibility on resource-constrained systems.

- **Incomplete Documentation of Tools:**

- While Python and GNU Radio are referenced for modular scripts, specific libraries, dependencies, or version compatibility issues are not addressed.
- Lack of detailed user manuals or tutorials for non-expert users, despite being listed as deliverables in Phase 5.

- **Cooperative Sensing Deferred:**

- Cooperative sensing with multiple HackRF units is mentioned as a future enhancement but lacks a preliminary framework or simulation results to validate its potential benefits.

- **Algorithmic Oversimplification**

- PSD estimation: Multitaper is robust, but computational cost at 16k FFT sizes is nontrivial; no trade-off analysis vs. Welch's method.
- Modulation recognition: Assumes clean separation of signals; no handling of overlapping transmissions (common in unlicensed bands).
- Noise floor estimation: FCME is cited, but performance degrades under non-stationary noise—a common real-world condition.